University of Niš
Faculty of Sciences and Mathematics
Department of Computer Science
Doctoral Academic Studies in Computer Science

Seminar paper
Course: Advanced Numerical Optimization (20.IDI16)

The BFGS and L-BFGS Methods

Mentor
Prof. Marko Miladinović

Student
Elvir Muslić
ORCID 0009-0006-4042-6550

Niš, October 2026

Abstract

Quasi-Newton methods minimize a smooth function of n variables without second derivatives: after every step, they correct an approximation of the inverse Hessian from the change of the gradient. This paper motivates the BFGS update by the secant equation, proves that it stays positive definite under the strong Wolfe conditions, and presents a line search for these conditions, the limited-memory variant L-BFGS with its two-loop recursion, and four convergence results. Our own implementations of gradient descent, BFGS and L-BFGS are compared on the Rosenbrock function and two test functions with up to 10^6 variables, with SciPy only as a check. On the Rosenbrock function, BFGS needs 38 iterations and gradient descent 5508. With five stored pairs, L-BFGS solves both test functions at $n = 10^6$, in 1104 and 239 iterations. Along SciPy's steps, our update reproduces SciPy's BFGS matrix to a relative difference of $3.0 \cdot 10^{-13}$.

Contents

1. Introduction 2
2. The BFGS Method 3
3. Limited-Memory BFGS 6
4. Convergence 7
5. Experiments 8
6. Conclusion 12

Code availability 13

References 13

1. Introduction

This paper is about the unconstrained problem

$$\min_{x \in \mathbb{R}^n} f(x), \tag{1.1}$$

where $f : \mathbb{R}^n \to \mathbb{R}$ is twice continuously differentiable. In words, (1.1) asks for a point x^* at which f is smallest.

Newton's method steps to the minimizer of the second-order Taylor model of f at x_k, $p_k = -\nabla^2 f(x_k)^{-1} \nabla f(x_k)$. Near a minimizer with a positive definite, Lipschitz continuous Hessian it converges quadratically [1, §3.3], but each step needs $n(n+1)/2$ second derivatives and an $O(n^3)$ linear solve.

Quasi-Newton methods replace $\nabla^2 f(x_k)^{-1}$ by a matrix H_k, so that the direction $-H_k \nabla f(x_k)$ costs one matrix-vector product, $O(n^2)$ operations, instead of a linear solve. They correct H_k after each step from gradients alone, also in $O(n^2)$ operations, and still converge superlinearly [1–3]. The best known is BFGS, by Broyden, Fletcher, Goldfarb and Shanno [4–7]. Its limited-memory variant L-BFGS [8, 9] stores a few vectors, not H_k, and handles millions of variables. The paper does four things.

(1) Section 2 presents the BFGS update, proves that it stays positive definite, and gives its algorithm.
(2) Section 3 derives L-BFGS and its two-loop recursion.

(3) Section 4 states four convergence results.
(4) Section 5 tests our code up to $n = 10^6$, checked against SciPy [10, 11].

2. The BFGS Method

2.1. **The secant condition.** Write $g_k = \nabla f(x_k)$; $\|\cdot\|$ is the Euclidean norm. A line search method moves from x_k along a direction p_k by a step length $\alpha_k > 0$, needed because the direction comes from a local model that can mislead far from x^*. A quasi-Newton method uses the Newton direction with $\nabla^2 f(x_k)^{-1}$ replaced by a symmetric positive definite matrix H_k:

$$x_{k+1} = x_k + \alpha_k p_k, \qquad p_k = -H_k\, g_k. \tag{2.1}$$

In words, the method moves along the negative gradient transformed by H_k, by a distance set by α_k; with $H_k = I$ it is gradient descent. Since $g_k^{\mathsf{T}} p_k = -g_k^{\mathsf{T}} H_k g_k < 0$ for $g_k \neq 0$, p_k is a *descent direction*: the slope of f along p_k is negative, so f decreases for small enough α_k.

The step length must decrease f enough, and it must not be too short, since tiny steps can decrease f at every iteration and still stall far from x^*. For constants $0 < c_1 < c_2 < 1$, the *strong Wolfe conditions* [1, (3.7)] are

$$f(x_k + \alpha_k p_k) \le f(x_k) + c_1\, \alpha_k\, g_k^{\mathsf{T}} p_k, \tag{2.2}$$

$$\left|\nabla f(x_k + \alpha_k p_k)^{\mathsf{T}} p_k\right| \le c_2\, \left|g_k^{\mathsf{T}} p_k\right|. \tag{2.3}$$

In words, the *sufficient decrease condition* (2.2) asks for at least a fraction c_1 of the decrease promised by the slope $g_k^{\mathsf{T}} p_k$, and the *strong curvature condition* (2.3) asks the slope along p_k to shrink to at most a fraction c_2 of its initial size, so that f neither still falls nor already rises steeply where the step ends. We use $c_1 = 10^{-4}$ and $c_2 = 0.9$, the usual values for quasi-Newton methods [1, Ch. 3].

The Hessian is not available, but each step reveals curvature through the pair

$$s_k = x_{k+1} - x_k = \alpha_k p_k, \qquad y_k = g_{k+1} - g_k. \tag{2.4}$$

In words, s_k is the step just taken and y_k the change of the gradient over it. By Taylor's theorem, $y_k = \nabla^2 f(x_{k+1})\, s_k + o(\|s_k\|)$, and $y_k = A s_k$ exactly for a quadratic with Hessian A. Asking the new matrix to reproduce this relation gives the *secant equation*, also called the secant condition,

$$H_{k+1}\, y_k = s_k. \tag{2.5}$$

In words, H_{k+1} must map the observed change of the gradient back to the step that caused it, as the inverse Hessian of a quadratic does.

A positive definite H_{k+1} can satisfy (2.5) only if $y_k^{\mathsf{T}} s_k = y_k^{\mathsf{T}} H_{k+1} y_k > 0$ (here $y_k \neq 0$, since $H_{k+1} y_k = s_k \neq 0$), so the update needs the *curvature condition*

$$s_k^{\mathsf{T}} y_k > 0. \tag{2.6}$$

In words, the slope of f along the step must grow from its start to its end, as it does for every strictly convex f; for a nonconvex f, the strong Wolfe conditions enforce it.

Lemma 2.1 (Wolfe steps give positive curvature)**.** *If $g_k^{\mathsf{T}} p_k < 0$ and $\alpha_k > 0$ satisfies* (2.3)*, then*

$$s_k^{\mathsf{T}} y_k \geq (1 - c_2)\, \alpha_k\, |g_k^{\mathsf{T}} p_k| > 0.$$

Proof. Since $g_k^{\mathsf{T}} p_k < 0$, condition (2.3) gives $\nabla f(x_{k+1})^{\mathsf{T}} p_k \geq -c_2 |g_k^{\mathsf{T}} p_k| = c_2\, g_k^{\mathsf{T}} p_k$. With (2.4) and $c_2 < 1$,

$$s_k^{\mathsf{T}} y_k = \alpha_k \big(\nabla f(x_{k+1})^{\mathsf{T}} p_k - g_k^{\mathsf{T}} p_k\big) \geq (1 - c_2)\, \alpha_k\, |g_k^{\mathsf{T}} p_k| > 0. \qquad \square$$

In words, every strong Wolfe step yields positive curvature, for any f.

2.2. **The BFGS update.** For $n > 1$ the secant equation imposes n conditions on the $n(n+1)/2$ entries of a symmetric H_{k+1}, so it does not determine it. BFGS takes the symmetric solution closest to H_k in a weighted Frobenius norm [1, §6.1], which keeps as much of H_k as the new pair allows. With $\rho_k = 1/(y_k^{\mathsf{T}} s_k)$ and $V_k = I - \rho_k\, y_k s_k^{\mathsf{T}}$, it is [1, (6.17)]

$$\begin{aligned} H_{k+1} &= V_k^{\mathsf{T}} H_k V_k + \rho_k\, s_k s_k^{\mathsf{T}} \\ &= \big(I - \rho_k\, s_k y_k^{\mathsf{T}}\big) H_k \big(I - \rho_k\, y_k s_k^{\mathsf{T}}\big) + \rho_k\, s_k s_k^{\mathsf{T}}. \end{aligned} \tag{2.7}$$

In words, the factors V_k remove from H_k its action on y_k, and the term $\rho_k s_k s_k^{\mathsf{T}}$ maps y_k to s_k, as (2.5) requires.

Theorem 2.2 (BFGS keeps H positive definite)**.** *Let H_k be symmetric positive definite and let $s_k^{\mathsf{T}} y_k > 0$. Then the matrix H_{k+1} of* (2.7) *(i) is symmetric, (ii) satisfies the secant equation* (2.5)*, and (iii) is positive definite.*

Proof. (i) Both $V_k^{\mathsf{T}} H_k V_k$ and $s_k s_k^{\mathsf{T}}$ are symmetric. (ii) Since $\rho_k\, s_k^{\mathsf{T}} y_k = 1$, we have $V_k y_k = y_k - \rho_k\, y_k (s_k^{\mathsf{T}} y_k) = 0$, and hence $H_{k+1} y_k = V_k^{\mathsf{T}} H_k (V_k y_k) + \rho_k\, s_k (s_k^{\mathsf{T}} y_k) = s_k$.

(iii) For $z \in \mathbb{R}^n$, $z^{\mathsf{T}} H_{k+1} z = (V_k z)^{\mathsf{T}} H_k (V_k z) + \rho_k\, (s_k^{\mathsf{T}} z)^2 \geq 0$, because H_k is positive definite and $\rho_k > 0$. If the sum vanishes, then $s_k^{\mathsf{T}} z = 0$ and $V_k z = 0$; but $V_k z = z - \rho_k\, y_k (s_k^{\mathsf{T}} z) = z$ when $s_k^{\mathsf{T}} z = 0$, so $z = 0$. $\square$

Lemma 2.1 and Theorem 2.2 give the key property of BFGS: if H_0 is symmetric positive definite and every step satisfies the strong Wolfe conditions, then every H_k is symmetric positive definite and every p_k is a descent direction.

Computed as written, (2.7) multiplies $n \times n$ matrices, $O(n^3)$ operations. Expanding the product gives the same matrix as a correction of rank two,

$$\begin{aligned} H_{k+1} &= H_k + s_k w_k^{\mathsf{T}} + w_k s_k^{\mathsf{T}}, \\ w_k &= \tfrac{1}{2} \rho_k \big(1 + \rho_k\, y_k^{\mathsf{T}} H_k y_k\big) s_k - \rho_k\, H_k y_k, \end{aligned} \tag{2.8}$$

which needs one matrix-vector product $H_k y_k$ and two outer products, $O(n^2)$ operations. In words, the update adds to H_k a symmetric matrix of rank at most two, built from s_k and $H_k y_k$.

2.3. **A computed update.** One update with small numbers shows Theorem 2.2 at work. Take $n = 2$, $H_k = I$, $s_k = (1, 0)^{\mathsf{T}}$ and $y_k = (2, 1)^{\mathsf{T}}$. Then $s_k^{\mathsf{T}} y_k = 2$, $\rho_k = 1/2$ and $V_k = I - \rho_k\, y_k s_k^{\mathsf{T}} = \left(\begin{smallmatrix} 0 & 0 \\ -1/2 & 1 \end{smallmatrix}\right)$, so (2.7) gives

$$H_{k+1} = V_k^{\mathsf{T}} V_k + \tfrac{1}{2}\, s_k s_k^{\mathsf{T}} = \begin{pmatrix} \frac{1}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix} + \begin{pmatrix} \frac{1}{2} & 0 \\ 0 & 0 \end{pmatrix} = \begin{pmatrix} \frac{3}{4} & -\frac{1}{2} \\ -\frac{1}{2} & 1 \end{pmatrix}.$$

The matrix is symmetric and satisfies the secant equation:

$$H_{k+1}\, y_k = \big(3/2 - 1/2,\ -1 + 1\big)^{\mathsf{T}} = (1,0)^{\mathsf{T}} = s_k.$$

Its trace $7/4$ and determinant $1/2$ give the eigenvalues $(7 \pm \sqrt{17})/8$, that is 0.3596 and 1.3904, so it is positive definite. The rank-two form (2.8), with $y_k^{\mathsf{T}} H_k y_k = 5$ and $w_k = (-1/8, -1/2)^{\mathsf{T}}$, gives the same matrix, as does the notebook with both formulas. The update has thus learned the one fact in the pair, that y_k maps to s_k, and stayed positive definite.

2.4. **The algorithm.** Algorithm 1 adds a stopping test, a start and a line search. A minimizer has a zero gradient, so the method stops when the gradient is small relative to its start, a test independent of the scale of f:

$$\|g_k\| \le \varepsilon \|g_0\|, \qquad \varepsilon = 10^{-6}. \tag{2.9}$$

In words, the gradient norm must fall by a factor of 10^6.

With $H_0 = I$ the first step is a gradient descent step, and the identity need not match the scale of the inverse Hessian, so before the first update H_k is replaced by $(y_k^{\mathsf{T}} s_k / y_k^{\mathsf{T}} y_k)\, I$ [1, (6.20)], the multiple of the identity that best satisfies $H y_k = s_k$ in the least-squares sense. The line search first tries $\alpha = 1$, the natural step of a Newton-like direction, and at $k = 0$ the step $\min(1, 1/\|g_0\|)$, of length at most 1 along $-g_0$.

Algorithm 2 [1, Algorithms 3.5 and 3.6] works on $\phi(\alpha) = f(x_k + \alpha p_k)$, whose slope $\phi'(\alpha) = \nabla f(x_k + \alpha p_k)^{\mathsf{T}} p_k$ is what (2.3) bounds; $\phi'(0) = g_k^{\mathsf{T}} p_k < 0$. Bracketing doubles the trial step until an interval must hold an acceptable step; zoom shrinks it, keeping in α_{lo} the best step satisfying (2.2). Line 10 is a safeguard: it bisects the bracket when the minimizer of the interpolant is not in its middle 80%, so the bracket keeps shrinking. The gradient is evaluated only where (2.2) holds, so f is evaluated at least as often as ∇f.

The code adds two safeguards against rounding errors and failed line searches; by Lemma 2.1 and Theorem 2.2, neither is needed in exact arithmetic when Algorithm 2 succeeds. First, lines 7–8 are skipped if $s_k^{\mathsf{T}} y_k \le 10^{-12} \|s_k\| \|y_k\|$. Second, if p_k is not a descent direction, the code discards H_k, restarts with $p_k = -g_k$ and scales again at the next update. If Algorithm 2 finds no acceptable step within 60 trials, or its bracket shrinks to rounding level, the code takes the best step found that satisfies (2.2), and stops if there is none.

Algorithm 1 The BFGS method

Input: starting point x_0, tolerance $\varepsilon = 10^{-6}$

1: $g_0 \leftarrow \nabla f(x_0)$, $H_0 \leftarrow I$, $k \leftarrow 0$
2: **while** $\|g_k\| > \varepsilon \|g_0\|$ **do** ▷ stopping test (2.9)
3: $p_k \leftarrow -H_k\, g_k$
4: $\alpha_k \leftarrow$ Algorithm 2, first trial step 1 (at $k = 0$: $\min(1, 1/\|g_0\|)$)
5: $x_{k+1} \leftarrow x_k + \alpha_k p_k$, $g_{k+1} \leftarrow \nabla f(x_{k+1})$
6: $s_k \leftarrow x_{k+1} - x_k$, $y_k \leftarrow g_{k+1} - g_k$
7: **if** this is the first update **then** $H_k \leftarrow (y_k^{\mathsf{T}} s_k / y_k^{\mathsf{T}} y_k)\, I$ ▷ scaling
8: $H_{k+1} \leftarrow H_k + s_k w_k^{\mathsf{T}} + w_k s_k^{\mathsf{T}}$ ▷ update (2.7) in the form (2.8)
9: $k \leftarrow k + 1$
10: **return** x_k

Algorithm 2 Line search for the strong Wolfe conditions

Input: $\phi(\alpha) = f(x_k + \alpha p_k)$ with $\phi'(0) < 0$, first trial step $\alpha > 0$

1: $\alpha_{\text{prev}} \leftarrow 0$
2: **loop** ▷ bracketing
3: **if** (2.2) fails at α **or** $\phi(\alpha) \geq \phi(\alpha_{\text{prev}})$ **then return** Zoom$(\alpha_{\text{prev}}, \alpha)$
4: **if** (2.3) holds at α **then return** α
5: **if** $\phi'(\alpha) \geq 0$ **then return** Zoom$(\alpha, \alpha_{\text{prev}})$
6: $\alpha_{\text{prev}} \leftarrow \alpha$, $\alpha \leftarrow 2\alpha$
7: **function** Zoom$(\alpha_{\text{lo}}, \alpha_{\text{hi}})$ ▷ α_{lo} satisfies (2.2)
8: **loop**
9: $\alpha \leftarrow$ minimizer of a cubic or quadratic interpolant of ϕ
10: **if** α is not in the middle 80% of the bracket **then** $\alpha \leftarrow (\alpha_{\text{lo}} + \alpha_{\text{hi}})/2$
11: **if** (2.2) fails at α **or** $\phi(\alpha) \geq \phi(\alpha_{\text{lo}})$ **then** $\alpha_{\text{hi}} \leftarrow \alpha$
12: **else if** (2.3) holds at α **then return** α
13: **else if** $\phi'(\alpha)(\alpha_{\text{hi}} - \alpha_{\text{lo}}) \geq 0$ **then** $\alpha_{\text{hi}} \leftarrow \alpha_{\text{lo}}$, $\alpha_{\text{lo}} \leftarrow \alpha$
14: **else** $\alpha_{\text{lo}} \leftarrow \alpha$

Gradient descent, our baseline, is Algorithm 1 without lines 7–8; for $k \geq 1$ it first tries $\alpha_{k-1}\, g_{k-1}^{\mathsf{T}} p_{k-1} / g_k^{\mathsf{T}} p_k$, the step whose predicted decrease $\alpha |g_k^{\mathsf{T}} p_k|$ equals that of the previous step [1, §3.5].

3. Limited-Memory BFGS

3.1. **Why limited memory.** BFGS stores H_k, n^2 numbers or $8n^2$ bytes: 8 MB at $n = 10^3$, 80 GB at $n = 10^5$ and 8 TB at $n = 10^6$, far beyond a workstation. L-BFGS keeps only the m latest pairs (s_i, y_i), $i = k-m, \ldots, k-1$, that is $2mn$ numbers, or 80 MB for $m = 5$ and $n = 10^6$. The oldest pair, which describes the curvature far behind the iterate, is dropped when a new one arrives [1, §7.2].

The matrix H_k is defined, but never formed, as the result of m updates (2.7) with these pairs, oldest first, applied to an initial matrix $H_k^0 = \gamma_k I$, chosen anew at every iteration, with [1, §7.2]

$$\gamma_k = \frac{s_{k-1}^{\mathsf{T}} y_{k-1}}{y_{k-1}^{\mathsf{T}} y_{k-1}}. \tag{3.1}$$

In words, γ_k is the scaling of Algorithm 1 for the latest pair, so H_k^0 fits the latest curvature and the unit step is usually accepted. By Lemma 2.1, every stored pair has $s_i^{\mathsf{T}} y_i > 0$, so $\gamma_k > 0$, and Theorem 2.2, applied m times, makes H_k positive definite.

3.2. **The two-loop recursion.** L-BFGS needs only $H_k g_k$, which Algorithm 3, the two-loop recursion of Nocedal [8], computes without forming H_k.

The loops apply the m updates (2.7) to a vector q. By the newest of them,

$$H_k q = V_{k-1}^{\mathsf{T}} \big(\tilde{H}\, V_{k-1} q\big) + a_{k-1} s_{k-1},$$
$$V_{k-1} q = q - a_{k-1} y_{k-1}, \qquad a_{k-1} = \rho_{k-1} s_{k-1}^{\mathsf{T}} q,$$

where $\tilde{H}$ is built in the same way from the $m-1$ older pairs; unrolling this m times gives the two loops.

Algorithm 3 The two-loop recursion [1, Algorithm 7.4]

Input: g_k; pairs (s_i, y_i) with $\rho_i = 1/(y_i^\mathsf{T} s_i)$, $i = k-m, \dots, k-1$; γ_k from (3.1)

1: $q \leftarrow g_k$
2: **for** $i = k-1, k-2, \dots, k-m$ **do** ▷ newest pair first
3: $\quad a_i \leftarrow \rho_i\, s_i^\mathsf{T} q, \;\; q \leftarrow q - a_i\, y_i$
4: $r \leftarrow \gamma_k\, q$ ▷ apply $H_k^0 = \gamma_k I$
5: **for** $i = k-m, \dots, k-1$ **do** ▷ oldest pair first
6: $\quad b \leftarrow \rho_i\, y_i^\mathsf{T} r, \;\; r \leftarrow r + (a_i - b)\, s_i$
7: **return** $r = H_k\, g_k$

- The first loop applies the right factors V_i, newest pair first: $V_i q = q - a_i y_i$, with $a_i = \rho_i s_i^\mathsf{T} q$; the terms $a_i s_i$ are kept for later.
- In the middle, $H_k^0 = \gamma_k I$ acts: $r = \gamma_k q$.
- The second loop applies the left factors V_i^T, oldest pair first, and adds the kept terms: $V_i^\mathsf{T} r + a_i s_i = r + (a_i - b)\, s_i$, with $b = \rho_i y_i^\mathsf{T} r$.

The notebook confirms this on random pairs ($n = 7$, $m = 6$), with a relative difference of $3.9 \cdot 10^{-16}$.

Each loop makes m passes with one inner product and one vector update of length n: about $4mn$ multiplications, $20n$ for $m = 5$, against n^2 for $H_k g_k$ in BFGS. With fewer than m pairs stored, the loops run over those.

3.3. **The L-BFGS algorithm.** L-BFGS is Algorithm 1 with two changes.
(1) Line 3 takes $p_k = -r$, with $r = H_k g_k$ from Algorithm 3; at $k = 0$ no pair is stored, and $p_0 = -g_0$.
(2) Lines 7 and 8 store the pair (s_k, y_k) instead of forming a matrix, and drop the oldest pair when more than m are stored.

Everything else is unchanged; a skipped update stores no pair, and a restart drops all pairs. We use $m = 5$, as small values of m often suffice [1, §7.2].

4. Convergence

The methods differ mainly in how fast the error $e_k = \|x_k - x^*\|$ falls. Convergence is linear if $e_{k+1} \le r\, e_k$, superlinear if $e_{k+1}/e_k \to 0$, quadratic if $e_{k+1} \le C e_k^2$, and R-linear if $e_k \le C r^k$, for some $r \in (0,1)$, $C > 0$ and all large k. In words, a linear method gains at least a fixed number of correct digits per iteration, a superlinear one ever more, a quadratic one about doubles them, and an R-linear one gains a fixed number on average.

Theorem 4.1 (Linear rate of gradient descent [1, §3.3])**.** *Let $f(x) = x^\mathsf{T} A x/2 - b^\mathsf{T} x$ with A symmetric positive definite, $\kappa = \lambda_{\max}(A)/\lambda_{\min}(A)$ and $\|z\|_A = (z^\mathsf{T} A z)^{1/2}$. Gradient descent with exact line searches satisfies $\|x_{k+1} - x^*\|_A \le \big((\kappa-1)/(\kappa+1)\big)\|x_k - x^*\|_A$.*

In words, on a quadratic, each step of gradient descent multiplies the error by at most $(\kappa-1)/(\kappa+1)$, which is close to 1 when A is ill conditioned. Near a minimizer of a smooth f, gradient descent with exact line searches obeys a similar bound, with $A = \nabla^2 f(x^*)$ and any constant slightly above $(\kappa-1)/(\kappa+1)$ [1, §3.3].

Theorem 4.2 (Global convergence of BFGS [1, Theorem 6.5])**.** *Let f be twice continuously differentiable and* uniformly convex*: the level set $\mathcal{L} =$*

$\{x : f(x) \le f(x_0)\}$ is convex, and $\mu\|z\|^2 \le z^\mathsf{T}\nabla^2 f(x)\, z \le M\|z\|^2$ for some $0 < \mu \le M$ and all $z \in \mathbb{R}^n$, $x \in \mathcal{L}$. Then, for every symmetric positive definite H_0, the iterates of BFGS with steps that satisfy (2.2) *and* (2.3) *converge to the minimizer x^* of f.*

In words, on a uniformly convex function, BFGS finds the minimizer from any starting point and any positive definite H_0.

Theorem 4.3 (Superlinear rate of BFGS [2], [1, Theorem 6.6])**.** *Let f be twice continuously differentiable, and let the BFGS iterates converge to a minimizer x^* with $\nabla^2 f(x^*)$ positive definite and $\nabla^2 f$ Lipschitz continuous near x^*. If $\sum_k \|x_k - x^*\| < \infty$ and the line search tries $\alpha_k = 1$ first, with $c_1 < 1/2$, then $x_k \to x^*$ superlinearly.*

In words, near the solution, the BFGS directions approach the Newton directions, which by Dennis and Moré [2] makes the unit step acceptable and the error ratios e_{k+1}/e_k tend to zero. The condition $\sum_k \|x_k - x^*\| < \infty$ asks the errors to fall fast enough to be summable; R-linear convergence, $e_k \le Cr^k$, is enough.

Theorem 4.4 (R-linear rate of L-BFGS [9])**.** *Let f satisfy the assumptions of Theorem 4.2. Then, for every memory $m \ge 1$, the iterates of L-BFGS with $H_k^0 = \gamma_k I$ from* (3.1) *and steps that satisfy* (2.2) *and* (2.3) *with $c_1 < 1/2$ converge to x^* R-linearly.*

In words, L-BFGS also converges on uniformly convex functions, but the theory promises only an R-linear rate, not the superlinear rate of Theorem 4.3.

Our BFGS and L-BFGS meet the line-search hypotheses of Theorems 4.2–4.4, with $c_1 = 10^{-4} < 1/2$, $\alpha = 1$ tried first for $k \ge 1$, and every step satisfying (2.2) and (2.3); our gradient descent uses Algorithm 2, not exact line searches, so Theorem 4.1 gives only a reference rate for it. Theorems 4.2 and 4.4 cover the uniformly convex TRIDIA and Raydan 1 (Section 5.2); the nonconvex Rosenbrock function is covered only near its minimizer, where $\nabla^2 f$ is positive definite, by Theorem 4.3. Section 5 measures these rates and costs up to $n = 10^6$.

5. Experiments

All results come from our own implementation of Algorithms 1–3 in Python with NumPy; SciPy [10] is only a check (Section 5.4). We use $c_1 = 10^{-4}$, $c_2 = 0.9$, the test (2.9) with $\varepsilon = 10^{-6}$ unless stated otherwise, and $m = 5$ pairs in L-BFGS; the Rosenbrock function starts from $x_0 = (-1.2, 1)$, TRIDIA and Raydan 1 from $x_0 = (1, \dots, 1)$. Iteration and evaluation counts are deterministic and do not depend on the machine's speed; times do. The times come from one execution of the notebook on one loaded machine with one BLAS thread, so the tables hold counts only, and times appear only in Figure 3 and as orders of magnitude.

5.1. The Rosenbrock function.

BFGS converges as Theorem 4.3 describes, with nearly Newton directions and unit steps, and gradient descent converges linearly, with error ratios near 1. The Rosenbrock function tests whether a method can follow a curved valley:

$$f(x) = 100\,(x_2 - x_1^2)^2 + (1 - x_1)^2, \qquad x^* = (1, 1).$$

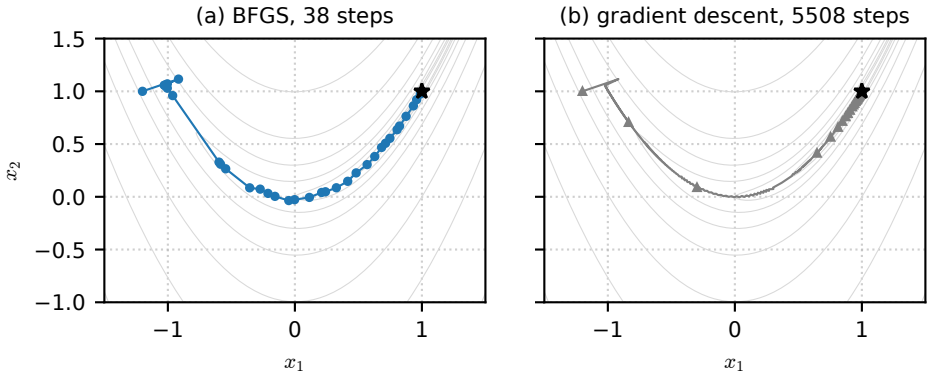


FIGURE 1. The Rosenbrock function from $x_0 = (-1.2, 1)$, $\varepsilon = 10^{-6}$: (a) BFGS, every iterate (38 steps); (b) gradient descent, a marker at every 100th of its 5508 iterates. The star is $x^* = (1, 1)$.

In words, f is small only near the parabola $x_2 = x_1^2$, and the factor 100 makes the valley narrow; f is not convex, and $\nabla^2 f(x^*)$ has the condition number $\kappa = 2508$. The experiment checks Lemma 2.1 at every step and measures the rates of Theorems 4.1 and 4.3.

Gradient descent needs 5508 iterations (5683 evaluations of f and 5607 of ∇f) and BFGS 38 (52 and 43), along the paths of Figure 1; L-BFGS needs 37 (48 and 41).

Every accepted step of the three methods satisfies (2.2) and (2.3) and has $s_k^{\mathsf{T}} y_k > 0$, so by Theorem 2.2 every BFGS matrix stays positive definite. Over the BFGS steps, $(f(x_k) - f(x_{k+1}))/(\alpha_k |g_k^{\mathsf{T}} p_k|) \geq 0.181 \geq c_1$, $|g_{k+1}^{\mathsf{T}} p_k| / |g_k^{\mathsf{T}} p_k| \leq 0.876 \leq c_2$ and $s_k^{\mathsf{T}} y_k \geq 5.5 \cdot 10^{-9}$. BFGS takes the unit step in its last 10 iterations, and its last matrix, rounded to four decimals, is close to the inverse Hessian at x^*:

$$H_k = \begin{pmatrix} 0.5 & 1 \\ 1 & 2.0051 \end{pmatrix}, \qquad \nabla^2 f(x^*)^{-1} = \begin{pmatrix} 0.5 & 1 \\ 1 & 2.005 \end{pmatrix}.$$

With $\varepsilon = 10^{-12}$, all three methods reach the test without stalling: gradient descent in 18168 iterations, BFGS and L-BFGS in 40. The error ratios e_{k+1}/e_k of BFGS and L-BFGS (Figure 2) stay near 1 for most iterations, then fall toward 0, not monotonically; the geometric means of their last four are 0.0095 and 0.0143. Theorem 4.4 promises L-BFGS only an R-linear rate, but here $m = 5$ exceeds $n = 2$, so the stored pairs span the whole space and L-BFGS can behave almost like BFGS.

The ratios of gradient descent stay near 1: their geometric mean over the last 1000 steps is 0.9990, close to $(\kappa - 1)/(\kappa + 1) = 0.9992$, and single ratios scatter around it, some above. This does not contradict Theorem 4.1, which assumes a quadratic, exact line searches and the norm $\|\cdot\|_A$, none of which holds here.

5.2. **TRIDIA and Raydan 1.** The pairs save most of the iterations of gradient descent, and neither quasi-Newton method always needs fewer. TRIDIA and Raydan 1 are two functions of Vilin [12], the course's MATLAB

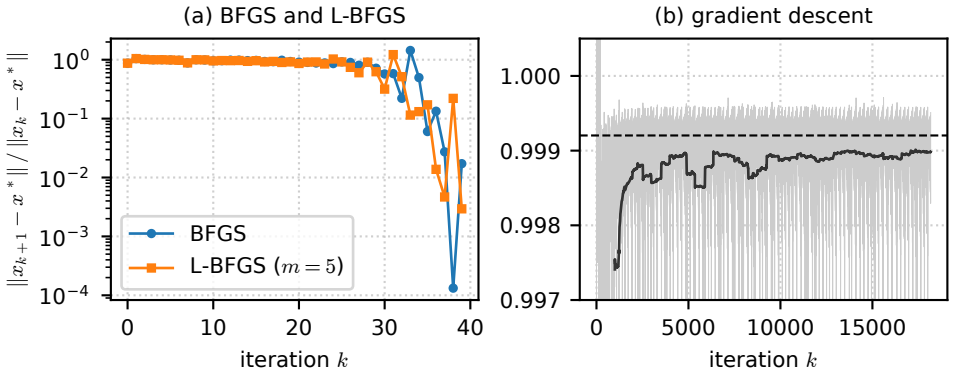


FIGURE 2. Error ratios $\|x_{k+1} - x^*\|/\|x_k - x^*\|$ on the Rosenbrock function, $\varepsilon = 10^{-12}$: (a) BFGS and L-BFGS ($m = 5$); (b) gradient descent: light grey, each ratio; solid, the geometric mean of the preceding 1000 ratios; dashed, $(\kappa - 1)/(\kappa + 1) = 0.9992$.

TABLE 1. TRIDIA and Raydan 1, $n = 100$, $x_0 = (1, \ldots, 1)$, $\varepsilon = 10^{-6}$: iterations, evaluations, and the final $\|g_k\|/\|g_0\|$.

function	method	iterations	f evals	∇f evals	$\|g_k\|/\|g_0\|$
TRIDIA	gradient descent	2547	2655	2605	$9.9 \cdot 10^{-7}$
	BFGS	73	79	75	$6.5 \cdot 10^{-7}$
	L-BFGS ($m = 5$)	135	138	137	$9.2 \cdot 10^{-7}$
Raydan 1	gradient descent	220	250	229	$9.9 \cdot 10^{-7}$
	BFGS	92	93	93	$9.2 \cdot 10^{-7}$
	L-BFGS ($m = 5$)	55	58	57	$8.9 \cdot 10^{-7}$

framework, from Andrei's collection [13]:

$$f(x) = (x_1 - 1)^2 + \sum_{i=2}^{n} i\,(2x_i - x_{i-1})^2 \quad \text{and} \quad f(x) = \sum_{i=1}^{n} \frac{i}{10}\left(e^{x_i} - x_i\right),$$

with the minimizers $x_i^* = 2^{1-i}$ and $x^* = 0$. In words, TRIDIA is a convex quadratic coupling neighboring variables, and Raydan 1 a weighted sum of convex functions of one variable. Both are uniformly convex in the sense of Theorem 4.2: TRIDIA has a constant positive definite Hessian, and the diagonal Hessian $(i/10)\,e^{x_i}$ of Raydan 1 is bounded above and away from 0 on the level set of x_0, where every x_i stays bounded. So Theorems 4.2 and 4.4 guarantee convergence; we measure its cost at $n = 100$.

BFGS needs the fewest iterations on TRIDIA and L-BFGS on Raydan 1, both far fewer than gradient descent (Table 1). All runs meet (2.9), and BFGS and L-BFGS use only a few more evaluations than iterations, so their line search nearly always accepts the first trial step. The two differ only in the memory of five pairs and in the scaling (3.1), renewed at every iteration; these runs do not show which difference makes L-BFGS slower on TRIDIA and faster on Raydan 1.

5.3. **Larger n.** L-BFGS keeps the memory and the work of an iteration linear in n while its iteration count grows slowly, so a million variables take seconds. By Section 3, an L-BFGS iteration needs memory and work of order mn, against n^2 for BFGS, and Theorem 4.4 holds for every n. L-BFGS runs at $n = 10^3$ to 10^6, gradient descent up to $n = 10^5$, capped at 10^4 iterations, and BFGS only at $n = 10^3$, as a dense H_k needs $8n^2$ bytes and $O(n^2)$ work per iteration.

L-BFGS meets the test at every n (Table 2): from $n = 10^3$ to 10^6, its iterations grow from 354 to 1104 on TRIDIA and from 121 to 239 on Raydan 1, while n grows a thousandfold. Gradient descent reaches the cap on TRIDIA and needs 983 to 2159 iterations on Raydan 1. The time per iteration (Figure 3) grows with slope about 2 for BFGS, the $O(n^2)$ cost of (2.8), and about 1 for L-BFGS and gradient descent (fits to the three largest n). In the run reported here, an iteration of BFGS at $n = 4000$ took tens of milliseconds, longer than one of L-BFGS at $n = 10^6$, about 10 ms, and L-BFGS solved Raydan 1 with $n = 10^6$ in seconds.

TABLE 2. Iterations from $x_0 = (1, \dots, 1)$ to meet (2.9) with $\varepsilon = 10^{-6}$. "cap": stopped at 10^4 iterations, at $\|g_k\|/\|g_0\| = 3.8 \cdot 10^{-6}$ ($n = 10^3$), $8.5 \cdot 10^{-6}$ ($n = 10^4$) and $1.1 \cdot 10^{-5}$ ($n = 10^5$); dash: not run.

function	method	$n = 10^3$	$n = 10^4$	$n = 10^5$	$n = 10^6$
TRIDIA	gradient descent	cap	cap	cap	–
	BFGS	241	–	–	–
	L-BFGS ($m = 5$)	354	862	1068	1104
Raydan 1	gradient descent	983	1985	2159	–
	BFGS	249	–	–	–
	L-BFGS ($m = 5$)	121	214	239	239

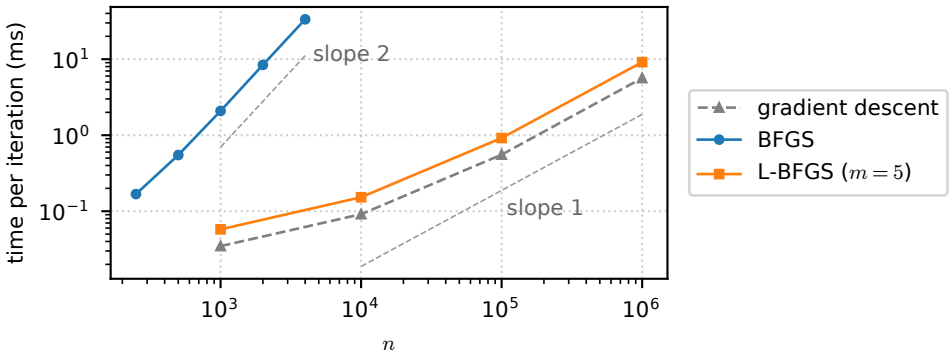


FIGURE 3. Time per iteration on TRIDIA, in milliseconds, the fastest of five runs of 30 iterations; dashed lines: slopes 1 and 2. Measured in one execution of the notebook on one loaded machine; read only the slopes and orders of magnitude.

5.4. **Checks against SciPy.** SciPy confirms our update to rounding error, and our solvers to the accuracy of the stopping test. SciPy's BFGS and L-BFGS-B [11], an L-BFGS method for bound constraints used here without bounds, are independent codes with their own line searches; they check (2.7) and our solvers. SciPy's BFGS takes 32 iterations on the Rosenbrock function, until $\|g_k\| \le 10^{-5}$. Applying (2.7) along all but its last step (SciPy does not update after the step that meets its test) from $H_0 = I$, as SciPy does not scale H_0, gives its last matrix to a relative difference of $3.0 \cdot 10^{-13}$ in the Frobenius norm.

Table 3 compares our BFGS with SciPy's at $n = 100$ and our L-BFGS with L-BFGS-B at $n = 1000$. SciPy's tolerances are set to apply (2.9), in the max norm for L-BFGS-B, with its test on the decrease of f switched off (`ftol` $= 0$). The iteration counts differ, since the codes differ in the line search, in H_0, the identity in SciPy's BFGS, and, for L-BFGS-B, in the norm of the test.

The final points agree to within $6.2 \cdot 10^{-5}$ at $n = 100$ and $3.7 \cdot 10^{-3}$ at $n = 1000$ in the max norm. That is the accuracy the test allows: (2.9) is relative to $\|g_0\|$, and at $n = 1000$ it already accepts our points at $7.6 \cdot 10^{-4}$ and $1.3 \cdot 10^{-3}$ from x^*, so two codes that both pass it can differ by about 10^{-3}.

TABLE 3. Our methods against SciPy's from $x_0 = (1, \dots, 1)$, all stopped by (2.9) with $\varepsilon = 10^{-6}$ (in the max norm for L-BFGS-B); final points x_{ours} and x_{SciPy}.

	iterations			
function	ours	SciPy	$\|x_{\text{ours}} - x_{\text{SciPy}}\|_\infty$	$\|x_{\text{ours}} - x^*\|_\infty$
BFGS against SciPy's BFGS, $n = 100$				
TRIDIA	73	109	$6.0 \cdot 10^{-5}$	$3.7 \cdot 10^{-6}$
Raydan 1	92	53	$6.2 \cdot 10^{-5}$	$4.6 \cdot 10^{-5}$
L-BFGS against SciPy's L-BFGS-B, both with $m = 5$, $n = 1000$				
TRIDIA	354	335	$3.7 \cdot 10^{-3}$	$7.6 \cdot 10^{-4}$
Raydan 1	121	136	$1.1 \cdot 10^{-3}$	$1.3 \cdot 10^{-3}$

6. Conclusion

(1) Every Rosenbrock step satisfies the strong Wolfe conditions with $s_k^{\mathsf{T}} y_k > 0$ (Lemma 2.1); BFGS needs 38 iterations, with error ratios falling toward zero, gradient descent 5508.
(2) At $n = 100$, BFGS and L-BFGS save most of the iterations of gradient descent; BFGS needs fewer on TRIDIA, L-BFGS on Raydan 1.
(3) L-BFGS solves both functions at $n = 10^6$, in 1104 and 239 iterations, with a time per iteration about linear in n; for dense BFGS it is about quadratic, and H_k would need 8 TB.
(4) SciPy confirms (2.7) to a relative difference of $3.0 \cdot 10^{-13}$, and our solvers to the accuracy of the test, about 10^{-3} at $n = 1000$.

The findings have limits: three test functions, one starting point each, one memory $m = 5$, and times from one execution on one loaded machine,

meaningful only as slopes and orders of magnitude. The counts are exact for this code; another line search, H_0 or norm in the test changes them (Section 5.4).

Code availability

The notebook `bfgs_project.ipynb`, in the folder `project`, implements Algorithms 1–3 and computes every number, table and figure in Sections 2.3, 3.2 and 5. It runs on Python 3.14.1 with NumPy 2.5.0, SciPy 1.18.1 and Matplotlib 3.11.0, pinned in `project/requirements.txt`; with these in a virtual environment `project/.venv`, the commands

```
cd project
.venv/bin/jupyter nbconvert --to notebook --execute --inplace bfgs_project.ipynb
```

run it; the run reported here took less than a minute. The public repository https://github.com/innicoder/bfgs-seminar holds this paper, its LaTeX source, the notebook with its outputs, and the figures.

References

[1] J. Nocedal and S. J. Wright. *Numerical Optimization*. 2nd ed. Springer Series in Operations Research and Financial Engineering. New York: Springer, 2006. DOI: 10.1007/978-0-387-40065-5.

[2] J. E. Dennis Jr. and J. J. Moré. "Quasi-Newton methods, motivation and theory". In: *SIAM Review* 19.1 (1977), pp. 46–89. DOI: 10.1137/1019005.

[3] M. Miladinović and P. Stanimirović. *Nelinearna optimizacija*. Niš: Univerzitet u Nišu, Prirodno-matematički fakultet, 2015.

[4] C. G. Broyden. "The convergence of a class of double-rank minimization algorithms 1. General considerations". In: *IMA Journal of Applied Mathematics* 6.1 (1970), pp. 76–90. DOI: 10.1093/imamat/6.1.76.

[5] R. Fletcher. "A new approach to variable metric algorithms". In: *The Computer Journal* 13.3 (1970), pp. 317–322. DOI: 10.1093/comjnl/13.3.317.

[6] D. Goldfarb. "A family of variable-metric methods derived by variational means". In: *Mathematics of Computation* 24.109 (1970), pp. 23–26. DOI: 10.1090/S0025-5718-1970-0258249-6.

[7] D. F. Shanno. "Conditioning of quasi-Newton methods for function minimization". In: *Mathematics of Computation* 24.111 (1970), pp. 647–656. DOI: 10.1090/S0025-5718-1970-0274029-X.

[8] J. Nocedal. "Updating quasi-Newton matrices with limited storage". In: *Mathematics of Computation* 35.151 (1980), pp. 773–782. DOI: 10.1090/S0025-5718-1980-0572855-7.

[9] D. C. Liu and J. Nocedal. "On the limited memory BFGS method for large scale optimization". In: *Mathematical Programming* 45.1–3 (1989), pp. 503–528. DOI: 10.1007/BF01589116.

[10] P. Virtanen, R. Gommers, T. E. Oliphant, et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature Methods* 17.3 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[11] R. H. Byrd, P. Lu, J. Nocedal, and C. Zhu. "A limited memory algorithm for bound constrained optimization". In: *SIAM Journal on Scientific Computing* 16.5 (1995), pp. 1190–1208. DOI: 10.1137/0916069.

[12] M. Miladinović and P. Živadinović. *Vilin: Unconstrained Numerical Optimization Application*. 2018. arXiv: 1812.10986 [cs.NA].

[13] N. Andrei. "An unconstrained optimization test functions collection". In: *Advanced Modeling and Optimization* 10.1 (2008), pp. 147–161.